

Proyecto 2 Entrega 4

Andre Jo 22199

2025-03-21

```
library(GGally)
library(dplyr)
library(tidyverse)
library(class)
library(ggplot2)
library(caret)
```

Debido a que los datos que observamos, tienen datos faltantes, se realiza un la media y el promedio de los datos para rellenar valores faltantes.

```
head(summary(datos))
```

```

##      Id      MSSubClass      MSZoning      LotFrontage
## Min.    :1461  Min.    : 20.00  Length:1459  Min.    : 21.00
## 1st Qu.:1826  1st Qu.: 20.00  Class :character  1st Qu.: 60.00
## Median :2190  Median : 50.00  Mode  :character  Median : 67.00
## Mean   :2190  Mean   : 57.38                      Mean   : 68.33
## 3rd Qu.:2554  3rd Qu.: 70.00                      3rd Qu.: 78.00
## Max.   :2919  Max.   :190.00                      Max.   :200.00
##      LotArea      Street      Alley      LotShape
## Min.    : 1470  Length:1459  Length:1459  Length:1459
## 1st Qu.: 7391  Class :character  Class :character  Class :character
## Median : 9399  Mode  :character  Mode  :character  Mode  :character
## Mean   : 9819
## 3rd Qu.:11518
## Max.   :56600
##      LandContour      Utilities      LotConfig      LandSlope
## Length:1459  Length:1459  Length:1459  Length:1459
## Class :character  Class :character  Class :character  Class :character
## Mode  :character  Mode  :character  Mode  :character  Mode  :character
##
##
##
##      Neighborhood      Condition1      Condition2      BldgType
## Length:1459  Length:1459  Length:1459  Length:1459
## Class :character  Class :character  Class :character  Class :character
## Mode  :character  Mode  :character  Mode  :character  Mode  :character
##
##
##
##      HouseStyle      OverallQual      OverallCond      YearBuilt
## Length:1459  Min.    : 1.000  Min.    :1.000  Min.    :1879
## Class :character  1st Qu.: 5.000  1st Qu.:5.000  1st Qu.:1953
## Mode  :character  Median : 6.000  Median :5.000  Median :1973
##                      Mean   : 6.079  Mean   :5.554  Mean   :1971
##                      3rd Qu.: 7.000  3rd Qu.:6.000  3rd Qu.:2001
##                      Max.   :10.000  Max.   :9.000  Max.   :2010
##      YearRemodAdd      RoofStyle      RoofMatl      Exterior1st
## Min.    :1950  Length:1459  Length:1459  Length:1459
## 1st Qu.:1963  Class :character  Class :character  Class :character
## Median :1992  Mode  :character  Mode  :character  Mode  :character
## Mean   :1984
## 3rd Qu.:2004
## Max.   :2010
##      Exterior2nd      MasVnrType      MasVnrArea      ExterQual
## Length:1459  Length:1459  Min.    : 0.00  Length:1459
## Class :character  Class :character  1st Qu.: 0.00  Class :character
## Mode  :character  Mode  :character  Median : 0.00  Mode  :character
##                      Mean   : 99.67
##                      3rd Qu.: 162.00
##                      Max.   :1290.00
##      ExterCond      Foundation      BsmtQual      BsmtCond
## Length:1459  Length:1459  Length:1459  Length:1459
## Class :character  Class:character  Class :character  Class :character

```

```

## Mode :character Mode :character Mode :character Mode :character
##
##
##
## BsmtExposure BsmtFinType1 BsmtFinSF1 BsmtFinType2
## Length:1459 Length:1459 Min. : 0.0 Length:1459
## Class :character Class :character 1st Qu.: 0.0 Class :character
## Mode :character Mode :character Median : 350.5 Mode :character
## Mean : 439.1
## 3rd Qu.: 752.0
## Max. :4010.0
## BsmtFinSF2 BsmtUnfSF TotalBsmtSF Heating
## Min. : 0.00 Min. : 0.0 Min. : 0 Length:1459
## 1st Qu.: 0.00 1st Qu.: 219.5 1st Qu.: 784 Class :character
## Median : 0.00 Median : 460.0 Median : 988 Mode :character
## Mean : 52.58 Mean : 554.2 Mean :1046
## 3rd Qu.: 0.00 3rd Qu.: 797.5 3rd Qu.:1304
## Max. :1526.00 Max. :2140.0 Max. :5095
## HeatingQC CentralAir Electrical X1stFlrSF
## Length:1459 Length:1459 Length:1459 Min. : 407.0
## Class :character Class :character Class :character 1st Qu.: 873.5
## Mode :character Mode :character Mode :character Median :1079.0
## Mean :1156.5
## 3rd Qu.:1382.5
## Max. :5095.0
## X2ndFlrSF LowQualFinSF GrLivArea BsmtFullBath
## Min. : 0 Min. : 0.000 Min. : 407 Min. :0.0000
## 1st Qu.: 0 1st Qu.: 0.000 1st Qu.:1118 1st Qu.:0.0000
## Median : 0 Median : 0.000 Median :1432 Median :0.0000
## Mean : 326 Mean : 3.543 Mean :1486 Mean :0.4339
## 3rd Qu.: 676 3rd Qu.: 0.000 3rd Qu.:1721 3rd Qu.:1.0000
## Max. :1862 Max. :1064.000 Max. :5095 Max. :3.0000
## BsmtHalfBath FullBath HalfBath BedroomAbvGr
## Min. :0.00000 Min. :0.000 Min. :0.0000 Min. :0.000
## 1st Qu.:0.00000 1st Qu.:1.000 1st Qu.:0.0000 1st Qu.:2.000
## Median :0.00000 Median :2.000 Median :0.0000 Median :3.000
## Mean :0.06511 Mean :1.571 Mean :0.3777 Mean :2.854
## 3rd Qu.:0.00000 3rd Qu.:2.000 3rd Qu.:1.0000 3rd Qu.:3.000
## Max. :2.00000 Max. :4.000 Max. :2.0000 Max. :6.000
## KitchenAbvGr KitchenQual TotRmsAbvGrd Functional
## Min. :0.000 Length:1459 Min. : 3.000 Length:1459
## 1st Qu.:1.000 Class :character 1st Qu.: 5.000 Class :character
## Median :1.000 Mode :character Median : 6.000 Mode :character
## Mean :1.042 Mean : 6.385
## 3rd Qu.:1.000 3rd Qu.: 7.000
## Max. :2.000 Max. :15.000
## Fireplaces FireplaceQu GarageType GarageYrBlt
## Min. :0.0000 Length:1459 Length:1459 Min. :1895
## 1st Qu.:0.0000 Class :character Class :character 1st Qu.:1960
## Median :0.0000 Mode :character Mode :character Median :1979
## Mean :0.5812 Mean :1978
## 3rd Qu.:1.0000 3rd Qu.:2001

```

```

## Max. :4.0000 Max. :2207
## GarageFinish GarageCars GarageArea GarageQual
## Length:1459 Min. :0.000 Min. : 0.0 Length:1459
## Class :character 1st Qu.:1.000 1st Qu.: 318.0 Class :character
## Mode :character Median :2.000 Median : 480.0 Mode :character
## Mean :1.766 Mean : 472.8
## 3rd Qu.:2.000 3rd Qu.: 576.0
## Max. :5.000 Max. :1488.0
## GarageCond PavedDrive WoodDeckSF OpenPorchSF
## Length:1459 Length:1459 Min. : 0.00 Min. : 0.00
## Class :character Class :character 1st Qu.: 0.00 1st Qu.: 0.00
## Mode :character Mode :character Median : 0.00 Median : 28.00
## Mean : 93.17 Mean : 48.31
## 3rd Qu.: 168.00 3rd Qu.: 72.00
## Max. :1424.00 Max. :742.00
## EnclosedPorch X3SsnPorch ScreenPorch PoolArea
## Min. : 0.00 Min. : 0.000 Min. : 0.00 Min. : 0.000
## 1st Qu.: 0.00 1st Qu.: 0.000 1st Qu.: 0.00 1st Qu.: 0.000
## Median : 0.00 Median : 0.000 Median : 0.00 Median : 0.000
## Mean : 24.24 Mean : 1.794 Mean : 17.06 Mean : 1.744
## 3rd Qu.: 0.00 3rd Qu.: 0.000 3rd Qu.: 0.00 3rd Qu.: 0.000
## Max. :1012.00 Max. :360.000 Max. :576.00 Max. :800.000
## PoolQC Fence MiscFeature MiscVal
## Length:1459 Length:1459 Length:1459 Min. : 0.00
## Class :character Class :character Class :character 1st Qu.: 0.00
## Mode :character Mode :character Mode :character Median : 0.00
## Mean : 58.17
## 3rd Qu.: 0.00
## Max. :17000.00
## MoSold YrSold SaleType SaleCondition
## Min. : 1.000 Min. :2006 Length:1459 Length:1459
## 1st Qu.: 4.000 1st Qu.:2007 Class :character Class :character
## Median : 6.000 Median :2008 Mode :character Mode :character
## Mean : 6.104 Mean :2008
## 3rd Qu.: 8.000 3rd Qu.:2009
## Max. :12.000 Max. :2010
## SalePrice
## Min. :135751
## 1st Qu.:168703
## Median :179209
## Mean :179184
## 3rd Qu.:186789
## Max. :281644

```

Separamos las variables a numericas.

1.Elabore un modelo de regresión usando K nearest Neighbors (KNN), el conjunto de entrenamiento y la variable respuesta SalesPrice. Prediga con el modelo y explique los resultados a los que llega. Asegúrese que los conjuntos de entrenamiento y prueba sean los mismos de las entregas anteriores para que los modelos sean comparables

```
train<-train[complete.cases(train),]  
k_vecinos <- round(sqrt(nrow(datos)),0)  
  
k_vecinos
```

```
## [1] 38
```

En total se observa que k es igual a 38.

```
parametros <- expand.grid(k = k_vecinos)  
modelo1 <- train(  
  SalePrice~.,  
  data=train,  
  method = "knn",  
  preProcess= c("center","scale","knnImpute"),  
  tuneGrid = parametros  
)  
predModelo1 <- predict(modelo1, newdata=datos)  
  
rmse <- RMSE(predModelo1, datos$SalePrice)  
mse <- rmse^2
```

MSE

```
mse
```

```
## [1] 2737344392
```

RMSE

```
rmse
```

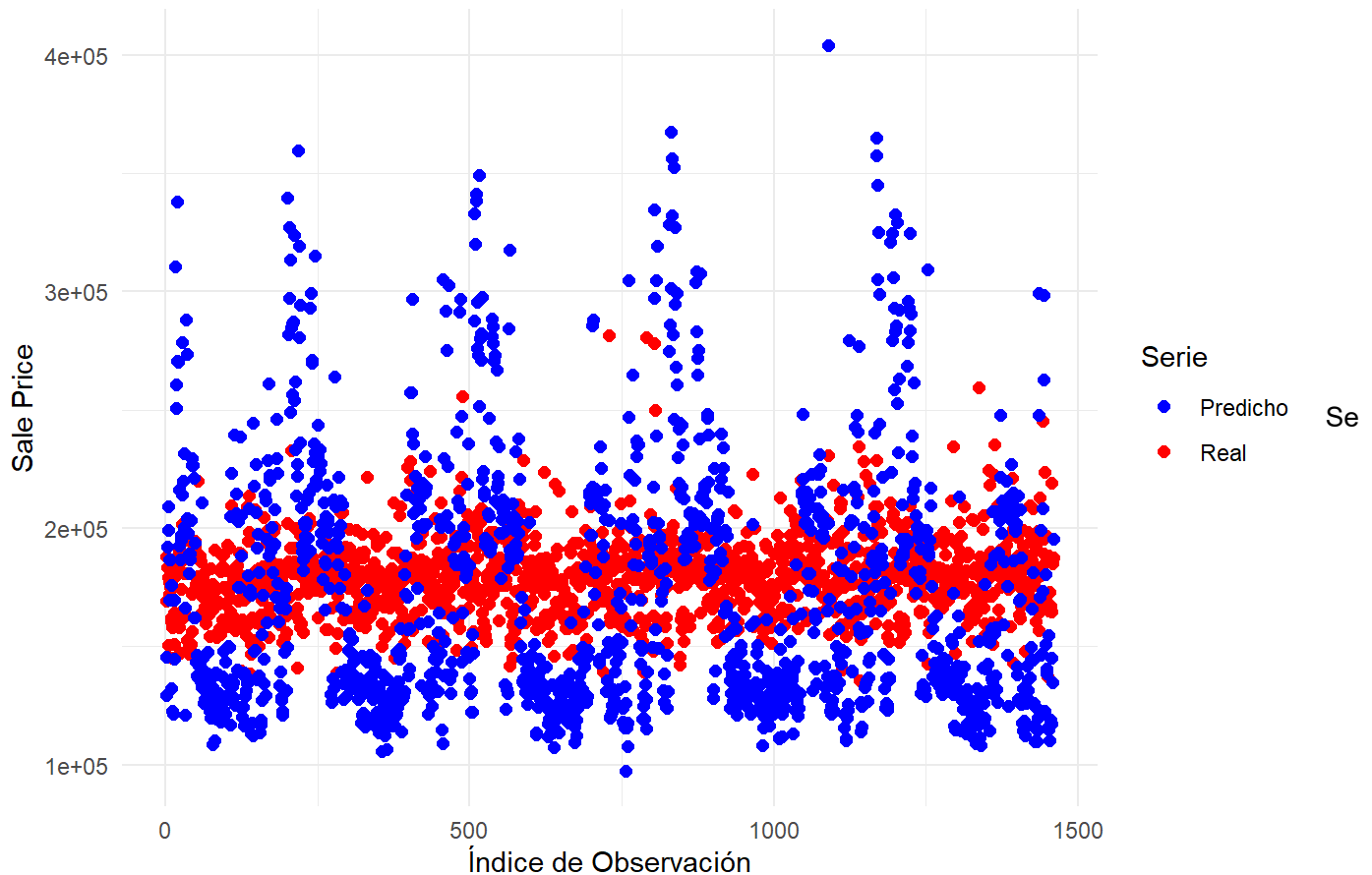
```
## [1] 52319.64
```

Se observa que el RMSE de el modelo de KNN es mejor a comparación de otros modelos como el modelo de Naive Bayes que tiene un RMSE de 70237.08 de acuerdo con la entrega pasada. Ahora con el modelo mejor que fue tuneado, se infiere a base de estos dos numeros (RMSE modelo tuneado 68855.19); se observa que el modelo de KNN es mucho mejor.

```
# Crear un dataframe con los valores reales y predichos
df_resultados <- data.frame(
  Índice = 1:length(datos$SalePrice), # Índice de cada observación
  Real = datos$SalePrice,
  Predicho = predModelo1
)

# Graficar ambas series
ggplot(df_resultados) +
  geom_point(aes(x = Índice, y = Real, color = "Real"), size = 2) + # Puntos rojos (valores reales)
  geom_point(aes(x = Índice, y = Predicho, color = "Predicho"), size = 2) + # Puntos azules (predicciones)
  scale_color_manual(values = c("Real" = "red", "Predicho" = "blue")) + # Definir colores
  labs(title = "Comparación: Valores Reales vs. Predicciones",
       x = "Índice de Observación",
       y = "Sale Price",
       color = "Serie") +
  theme_minimal()
```

Comparación: Valores Reales vs. Predicciones



observa en la gráfica que hay algunos puntos que el modelo no pudo predecir aunque no observamos que los datos estén sobreajustados como en los otros modelos vistos anteriormente. Por otro lado en los modelos anteriores se ve que se relaciona con el modelo de random forest.

8. Haga un modelo usando validación cruzada, compare los resultados de este con los del modelo anterior.
¿Cuál funcionó mejor?

```
trctrl <- trainControl(method = "repeatedcv",
                      number = 10,
                      repeats = 3)

modelo2 <- train(
  SalePrice~.,
  data=train,
  method = "knn",
  trControl = trctrl,
  preProcess= c("center","scale"),
  tuneGrid = parametros
)
predModelo2 <- predict(modelo2, newdata=datos)
```

```
RMSE(predModelo2, datos$SalePrice)
```

```
## [1] 52319.64
```

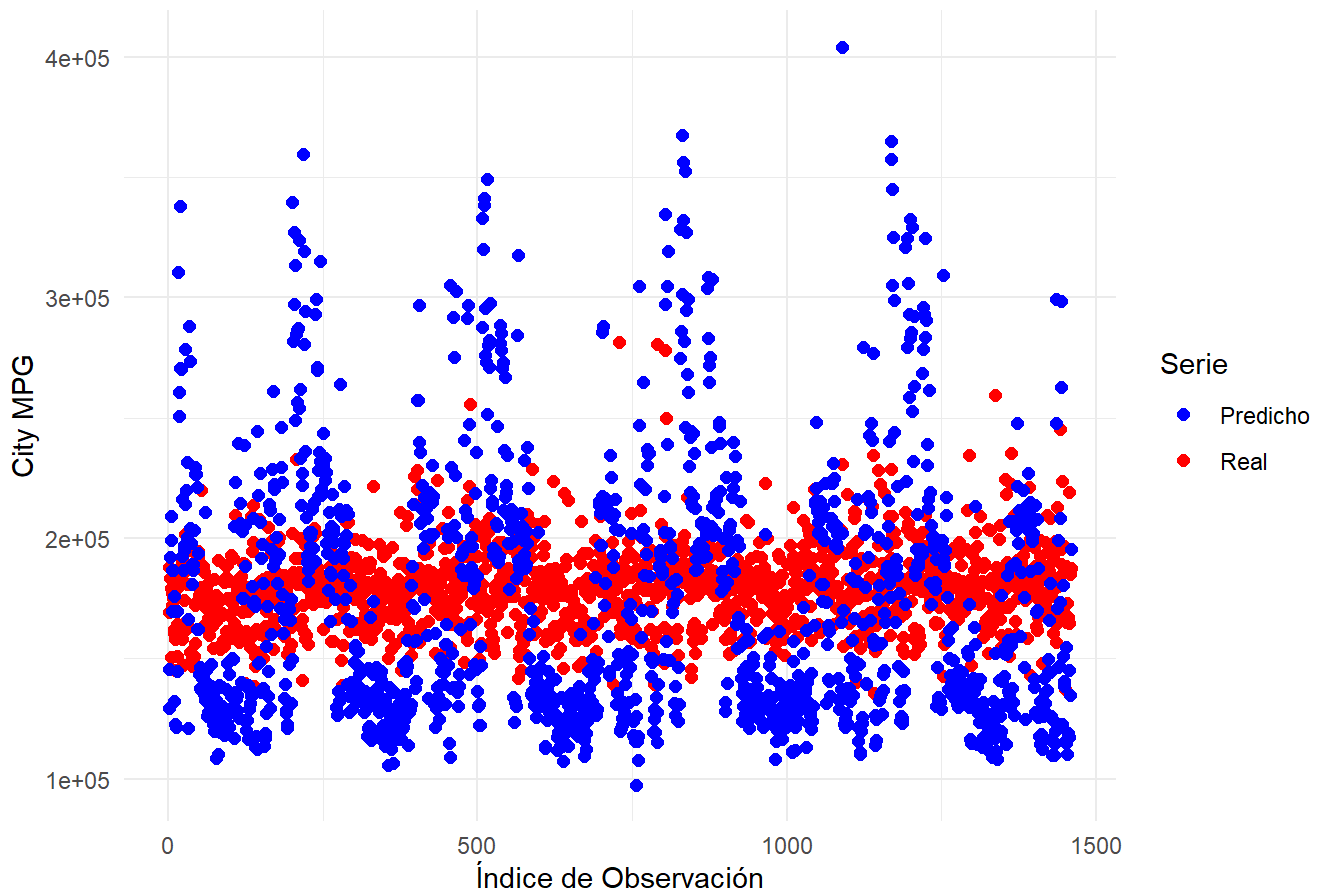
El RMSE es de 52319.64, es el mismo que el modelo de knn utilizado donde k es igual a la raíz cuadrada del número de filas (38).

En el siguiente gráfico se muestran los valores predichos y los reales.

```
df_resultados <- data.frame(
  Índice = 1:length(datos$SalePrice), # Índice de cada observación
  Real = datos$SalePrice,
  Predicho = predModelo2
)

# Graficar ambas series
ggplot(df_resultados) +
  geom_point(aes(x = Índice, y = Real, color = "Real"), size = 2) + # Puntos rojos (valores reales)
  geom_point(aes(x = Índice, y = Predicho, color = "Predicho"), size = 2) + # Puntos azules (predicciones)
  scale_color_manual(values = c("Real" = "red", "Predicho" = "blue")) + # Definir colores
  labs(title = "Comparación: Valores Reales vs. Predicciones",
       x = "Índice de Observación",
       y = "City MPG",
       color = "Serie") +
  theme_minimal()
```

Comparación: Valores Reales vs. Predicciones



Observamos que no tiene sobreajustes de datos y además observamos que con validación cruzada, tiene casi los mismos datos predichos que en la gráfica del modelo anterior. Ahora con la eficiencia del algoritmo se podría observar si es mejor el modelo con o sin validación cruzada.

9. Tanto para los modelos de regresión como de clasificación, pruebe con varios valores de los hiperparámetros ¿Qué parámetros pueden tunearse en un KNN?, use el mejor modelo del tuneo, ¿Mejoraron los resultados usando el mejor modelo ahora? Explique

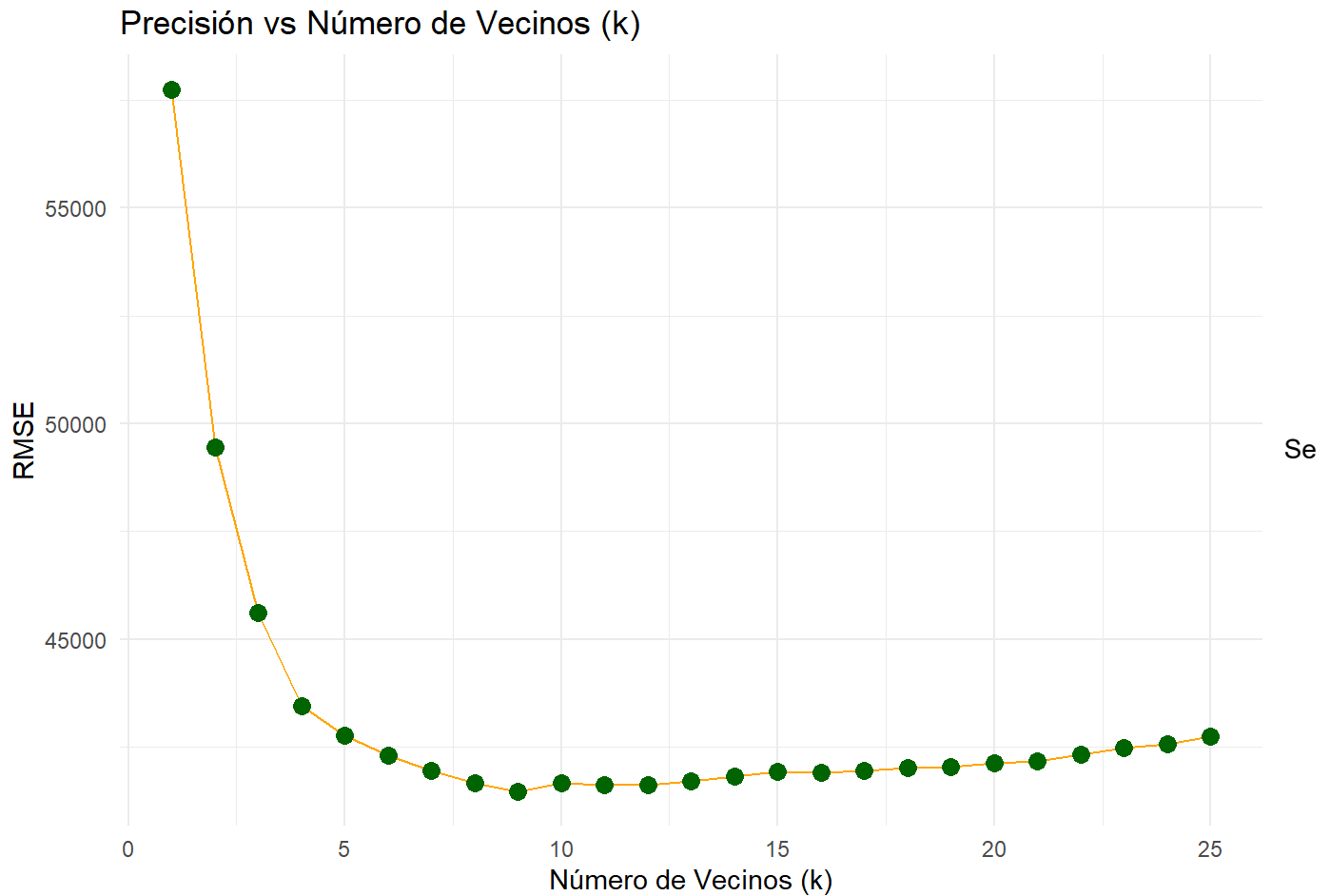
Se puede realizar el cambio de valor K, necesita tener validación cruzada e incluso preprocesar los datos.

Tuneando varios valores de k

```
parametros <- expand.grid(k = seq(1,25))
modelo3 <- train(
  SalePrice~.,
  data=train,
  method = "knn",
  trControl = trctrl,
  preProcess= c("center","scale"),
  tuneGrid = parametros
)
predModelo3 <- predict(modelo3, newdata=datos)
```

El mejor valor de k


```
ggplot(modelo3$results, aes(x = k, y = RMSE)) +
  geom_line(color = "orange") + # Línea azul
  geom_point(size = 3, color = "darkgreen") + # Puntos rojos
  labs(title = "Precisión vs Número de Vecinos (k)",
        x = "Número de Vecinos (k)",
        y = "RMSE") +
  theme_minimal() # Tema limpio
```

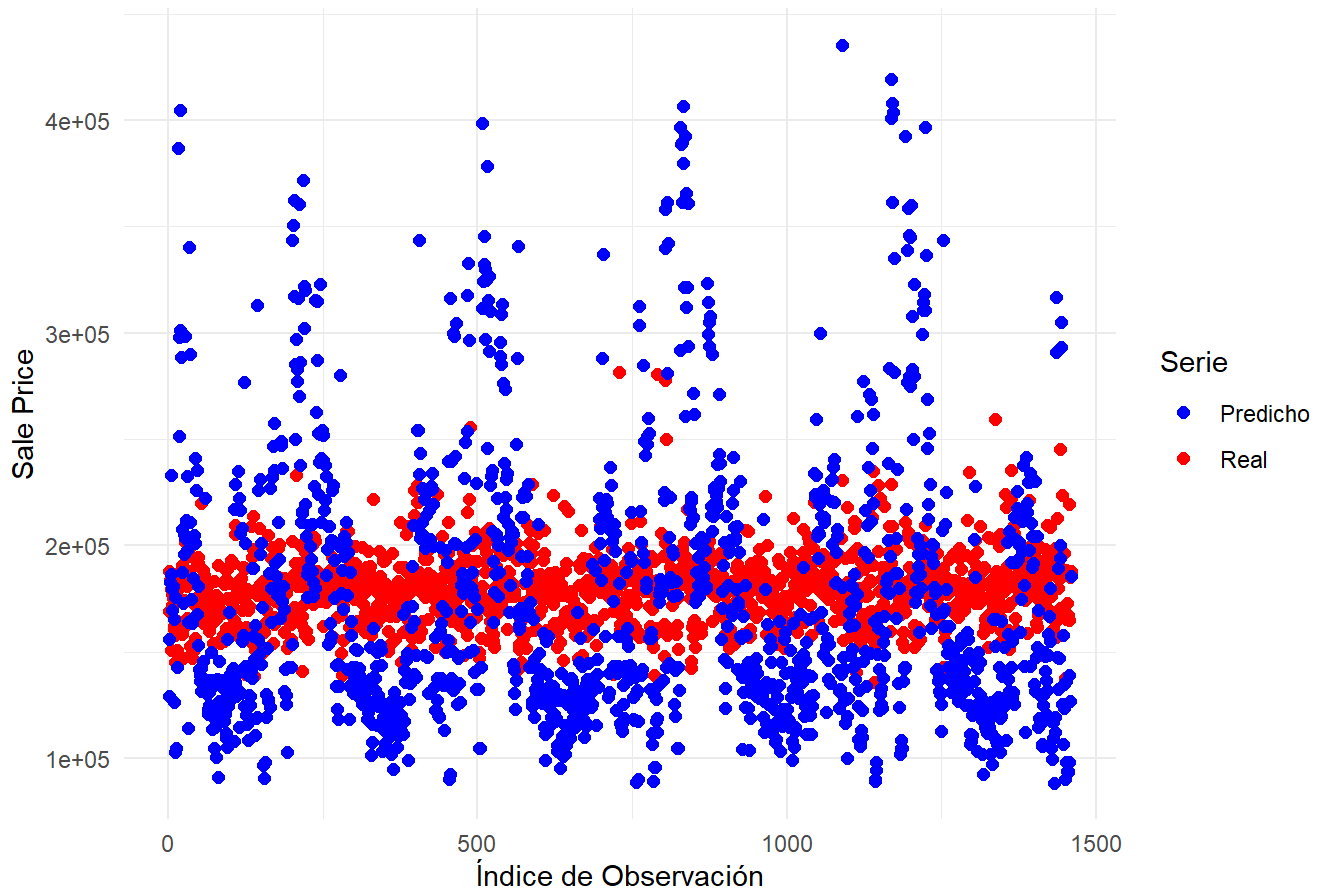


observa que el mejor vecino de acuerdo con la gráfica es 9 u 11.

```
df_resultados <- data.frame(
  Índice = 1:length(datos$SalePrice), # Índice de cada observación
  Real = datos$SalePrice,
  Predicho = predModelo3
)

# Graficar ambas series
ggplot(df_resultados) +
  geom_point(aes(x = Índice, y = Real, color = "Real"), size = 2) + # Puntos rojos (valores reales)
  geom_point(aes(x = Índice, y = Predicho, color = "Predicho"), size = 2) + # Puntos azules (predicciones)
  scale_color_manual(values = c("Real" = "red", "Predicho" = "blue")) + # Definir colores
  labs(title = "Comparación: Valores Reales vs. Predicciones",
       x = "Índice de Observación",
       y = "Sale Price",
       color = "Serie") +
  theme_minimal()
```

Comparación: Valores Reales vs. Predicciones



```
rmse <- RMSE(predModelo3, datos$SalePrice)
rmse
```

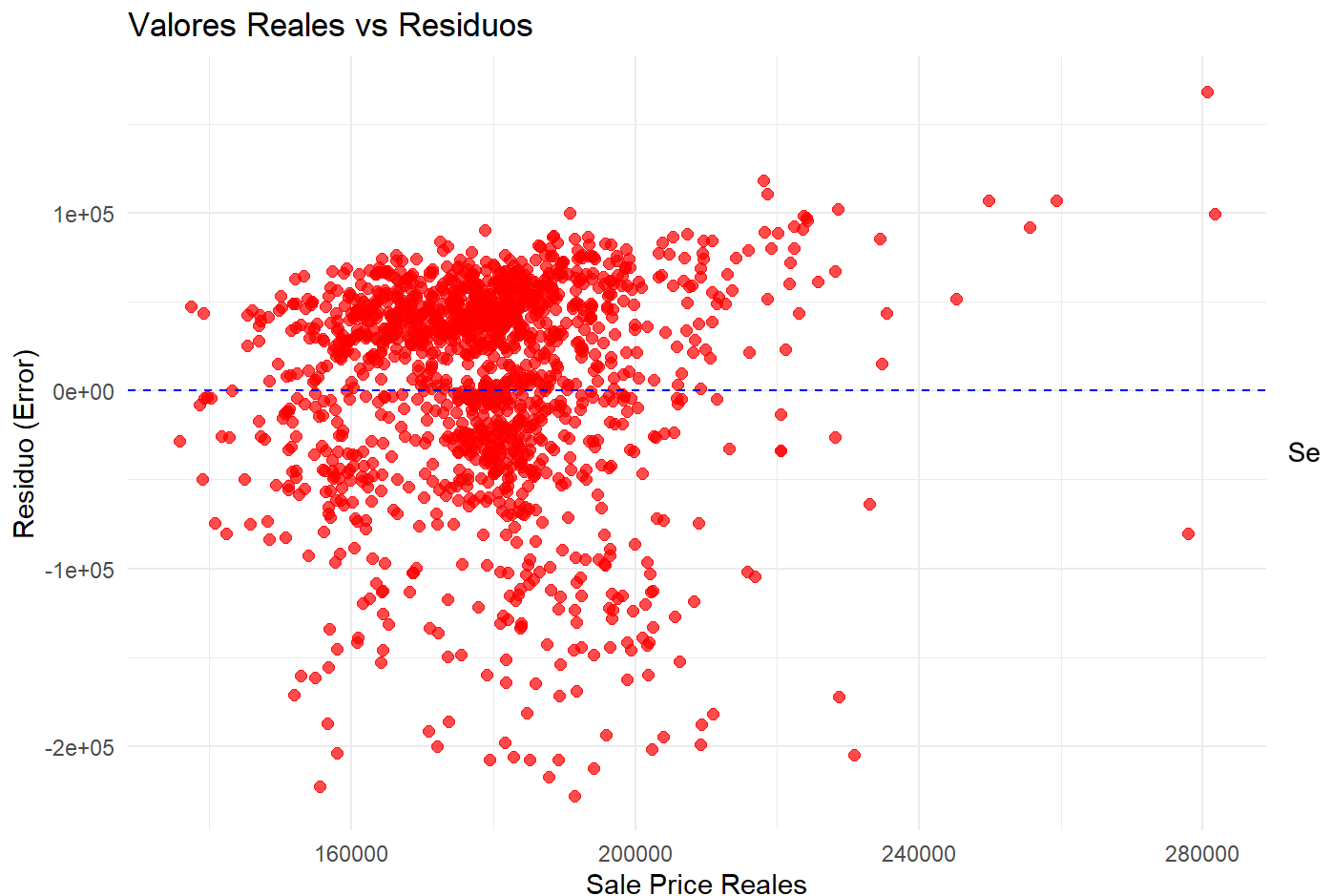
```
## [1] 60844.42
```

Se observa que el rmse al realizar diferentes configuraciones como realizar validación cruzada, preprocesamiento de datos e incluso cambiando el valor de k observamos que no es mejor a comparación del modelo 1 (k = 38). El modelo 1 tiene un rmse de 5319.64 que mucho mejor que este modelo (60844.42).

```
residuos <- datos$SalePrice - predModelo3

# Crear un dataframe con los valores reales y residuos
df_resultados <- data.frame(
  Real = datos$SalePrice,
  Residuo = residuos
)

# Graficar valores reales vs residuos
ggplot(df_resultados, aes(x = Real, y = Residuo)) +
  geom_point(color = "red", size = 2, alpha = 0.7) + # Puntos rojos
  geom_hline(yintercept = 0, linetype = "dashed", color = "blue") + # Línea en y = 0
  labs(title = "Valores Reales vs Residuos",
       x = "Sale Price Reales",
       y = "Residuo (Error)") +
  theme_minimal()
```



observa que la dispersión de los puntos rojos no es uniforme a lo largo del eje X. En la zona de precios de venta más bajos (160,000 - 200,000), los residuos tienden a ser más dispersos, mientras que en la zona de precios más altos, la dispersión es menor. Por lo cual se puede inferir que el modelo tiene una varianza de error no constante.

10. Compare la eficiencia del algoritmo con el resultado obtenido con el árbol de decisión (el de clasificación), el modelo de random forest y el de naive bayes que hizo en las entregas pasadas. ¿Cuál es mejor para predecir? ¿Cuál se demoró más en procesar?

Eficiencia del modelo donde se utiliza la raíz cuadrada de filas donde es 38

```
parametros <- expand.grid(k = k_vecinos)
system.time({
  modelo1 <- train(
    SalePrice~.,
    data=train,
    method = "knn",
    preProcess= c("center","scale","knnImpute"),
    tuneGrid = parametros
  )
})
```

```
##      user  system elapsed
##    9.30    0.39    9.81
```

Eficiencia del modelo utilizado con validación cruzada

```
trctrl <- trainControl(method = "repeatedcv",
                        number = 10,
                        repeats = 3)
system.time({
  modelo2 <- train(
    SalePrice~.,
    data=train,
    method = "knn",
    trControl = trctrl,
    preProcess= c("center","scale"),
    tuneGrid = parametros
  )
})
```

```
##      user  system elapsed
##    4.23    0.37    4.72
```

Eficiencia del modelo utilizando diferentes hiperparámetros

```
parametros <- expand.grid(k = seq(1,25))
system.time({
modelo3 <- train(
  SalePrice~.,
  data=train,
  method = "knn",
  trControl = trctrl,
  preProcess= c("center","scale"),
  tuneGrid = parametros
})
```

```
##      user  system elapsed
## 155.92   12.27   172.14
```

Se observa que el modelo con más hiperparametros puede tardar más a comparación de los dos modelos que estan cerca en terminos de tiempo de ejecución. Por otro lado observamos que los otros algoritmo como random forest y Naive Bayes observamos que el algoritmo más eficiente sigue siendo Naive Bayes. Por lo tanto inferimos que el modelo que es mejor para realizar producciones es utilizar el algoritmo de knn donde k es la raiz cuadrada del numero de filas donde tiene el mejor rmse (60844.42). Ahora en terminos de eficiencia seria mejor utilizar el modelo de Naive Bayes. Pero si se necesita un modelo que es adecuado con el tiempo de ejecución y una buena predicción, utilizaria el algoritmo de Random Forest.